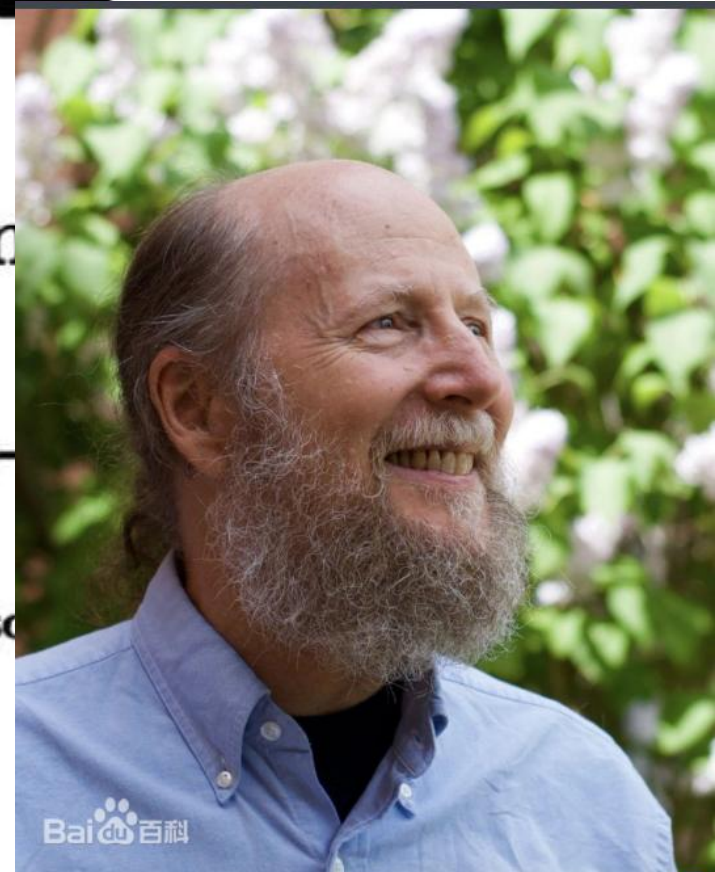


Policy Gradient Methods for Reinforcement Learning with Function Approximation

Richard S. Sutton, David McAllester, Satinder Singh, Yishay Mansour
AT&T Labs – Research, 180 Park Avenue, Florham Park, NJ 07932

Abstract



理查德·萨顿(Richard Sutton,图灵奖得主,强化学习之父,现任阿尔伯塔大学计算机科学教授)

Function approximation is essential to reinforcement learning, but the standard approach of approximating a value function and determining a policy from it has so far proven theoretically intractable. In this paper we explore an alternative approach in which the policy is explicitly represented by its own function approximator, independent of the value function, and is updated according to the gradient of expected reward with respect to the policy parameters. Williams's REINFORCE method and actor-critic methods are examples of this approach. Our main new result is to show that the gradient can be written in a form suitable for estimation from experience aided by an approximate action-value or advantage function. Using this result, we prove for the first time that a version of policy iteration with arbitrary differentiable function approximation is convergent to a locally optimal policy.

Large applications of reinforcement learning (RL) require the use of generalizing function approximators such as neural networks, decision-trees, or instance-based methods. The dominant approach for the last decade has been the *value-function* approach, in which all function approximation effort goes into estimating a value function, with the action-selection policy represented implicitly as the “greedy” policy with respect to the estimated values (e.g., as the policy that selects in each state the action with highest estimated value). The value-function approach has worked well in many applications, but has several limitations. First, it is oriented toward finding deterministic policies, whereas the optimal policy is often stochastic, selecting different actions with specific probabilities (e.g., see Singh, Jaakkola, and Jordan, 1994). Second, an arbitrarily small change in the estimated value of an action can cause it to be, or not be, selected. Such discontinuous changes have been identified as a key obstacle to establishing convergence assurances for algorithms following the value-function approach (Bertsekas and Tsitsiklis, 1996). For example, Q-learning, Sarsa, and dynamic programming methods have all been shown unable to converge to any policy for simple MDPs and simple function approximators (Gordon, 1995, 1996; Baird, 1995; Tsitsiklis and van Roy, 1996; Bertsekas and Tsitsiklis, 1996). This can occur even if the best approximation is found at each step before changing the policy, and whether the notion of “best” is in the mean-squared-error sense or the slightly different senses of residual-gradient, temporal-difference, and dynamic-programming methods.

In this paper we explore an alternative approach to function approximation in RL.

Rather than approximating a value function and using that to compute a deterministic policy, we approximate a stochastic policy directly using an independent function approximator with its own parameters. For example, the policy might be represented by a neural network whose input is a representation of the state, whose output is action selection probabilities, and whose weights are the policy parameters. Let θ denote the vector of policy parameters and ρ the performance of the corresponding policy (e.g., the average reward per step). Then, in the *policy gradient* approach, the policy parameters are updated approximately proportional to the gradient:

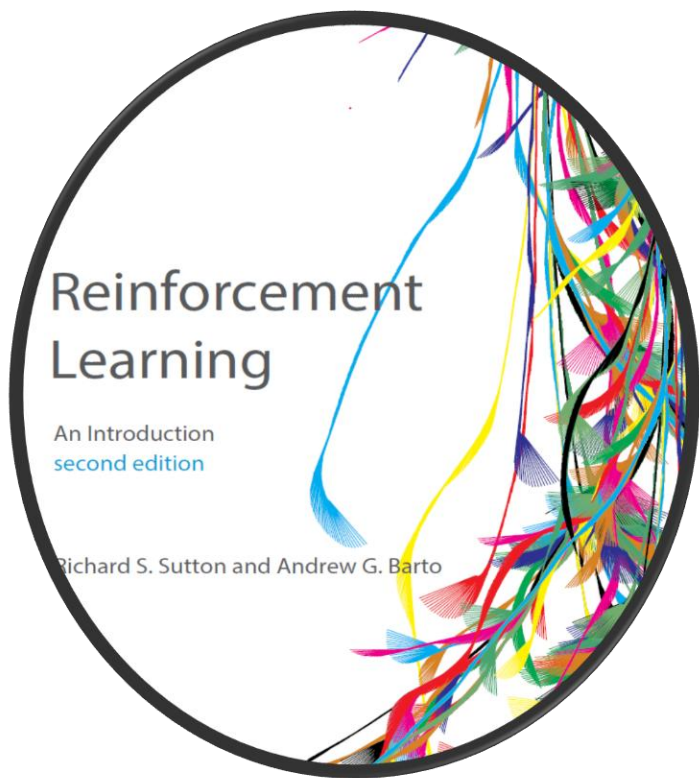
$$\Delta\theta \approx \alpha \frac{\partial \rho}{\partial \theta}, \quad (1)$$

where α is a positive-definite step size. If the above can be achieved, then θ can usually be assured to converge to a locally optimal policy in the performance measure ρ . Unlike the value-function approach, here small changes in θ can cause only small changes in the policy and in the state-visitation distribution.

In this paper we prove that an unbiased estimate of the gradient (1) can be obtained from experience using an approximate value function satisfying certain properties. Williams's (1988, 1992) REINFORCE algorithm also finds an unbiased estimate of the gradient, but without the assistance of a learned value function. REINFORCE learns much more slowly than RL methods using value functions and has received relatively little attention. Learning a value function and using it to reduce the variance of the gradient estimate appears to be essential for rapid learning. Jaakkola, Singh and Jordan (1995) proved a result very similar to ours for the special case of function approximation corresponding to tabular POMDPs. Our result strengthens theirs and generalizes it to arbitrary differentiable function approximators. Konda and Tsitsiklis (in prep.) independently developed a very similar result to ours. See also Baxter and Bartlett (in prep.) and Marbach and Tsitsiklis (1998).

Our result also suggests a way of proving the convergence of a wide variety of algorithms based on “actor-critic” or policy-iteration architectures (e.g., Barto, Sutton, and Anderson, 1983; Sutton, 1984; Kimura and Kobayashi, 1998). In this paper we take the first step in this direction by proving for the first time that a version of policy iteration with general differentiable function approximation is convergent to a locally optimal policy. Baird and Moore (1999) obtained a weaker but superficially similar result for their VAPS family of methods. Like policy-gradient methods, VAPS includes separately parameterized policy and value functions updated by gradient methods. However, VAPS methods do not climb the gradient of performance (expected long-term reward), but of a measure combining performance and value-function accuracy. As a result, VAPS does not converge to a locally optimal policy, except in the case that no weight is put upon value-function accuracy, in which case VAPS degenerates to REINFORCE. Similarly, Gordon’s (1995) fitted value iteration is also convergent and value-based, but does not find a locally optimal policy.

沿着性能曲面的梯度方向向上攀登（即朝着使性能最大化的方向更新参数）



第七章 策略梯度 (Policy Gradient)

何新卫

华中农业大学信息学院



回顾

□ 估计价值目标

- 估计状态价值 $v_{\pi}(s)$, for all $s \in \mathcal{S}$
- 估计状态-动作价值 $q_{\pi}(s, a)$, for all $s \in \mathcal{S}, a \in \mathcal{A}$

□ 方法介绍:

- 动态规划: 已知确定环境
- 蒙特卡洛方法: 已知或未知环境, 使用经验数据估算价值
- TD方法: 融合了动态规划和蒙特卡洛思想

□ 策略迭代 (Policy iteration) 找到最优价值函数

智能体在未知/已知环境中, 当计算出了最优价值函数后, 仍然无法直接做出决策。而是需要提取对应的最优策略。

基于价值的强化学习

$$\pi_*(a|s) = \operatorname{argmax}_{a \in \mathcal{A}(s)} \underline{q_*(s, a)}, \text{ for all } s \in \mathcal{S}$$

最优动作价值函数

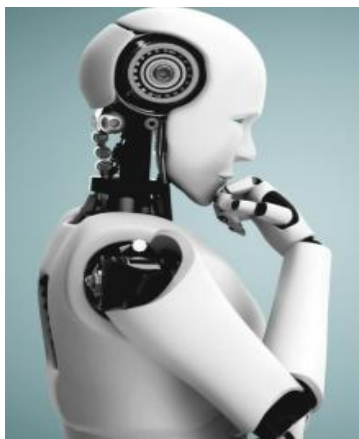
$$\begin{bmatrix} q_*(s_1, a_1) & \cdots & q_*(s_1, a_{|\mathcal{A}|}) \\ \vdots & \ddots & \vdots \\ q_*(s_{|\mathcal{S}|}, a_1) & \cdots & q_*(s_{|\mathcal{S}|}, a_{|\mathcal{A}|}) \end{bmatrix} \xrightarrow[\text{按行argmax}]{\text{最优策略提取}} \pi_*(a|s) = \begin{bmatrix} \pi_*(a|s_1) \\ \pi_*(a|s_2) \\ \cdots \\ \pi_*(a|s_{|\mathcal{S}|}) \end{bmatrix} = \begin{bmatrix} a_{j_1} \\ a_{j_2} \\ \cdots \\ a_{j_{|\mathcal{S}|}} \end{bmatrix}$$

最优动作价值函数 q_*

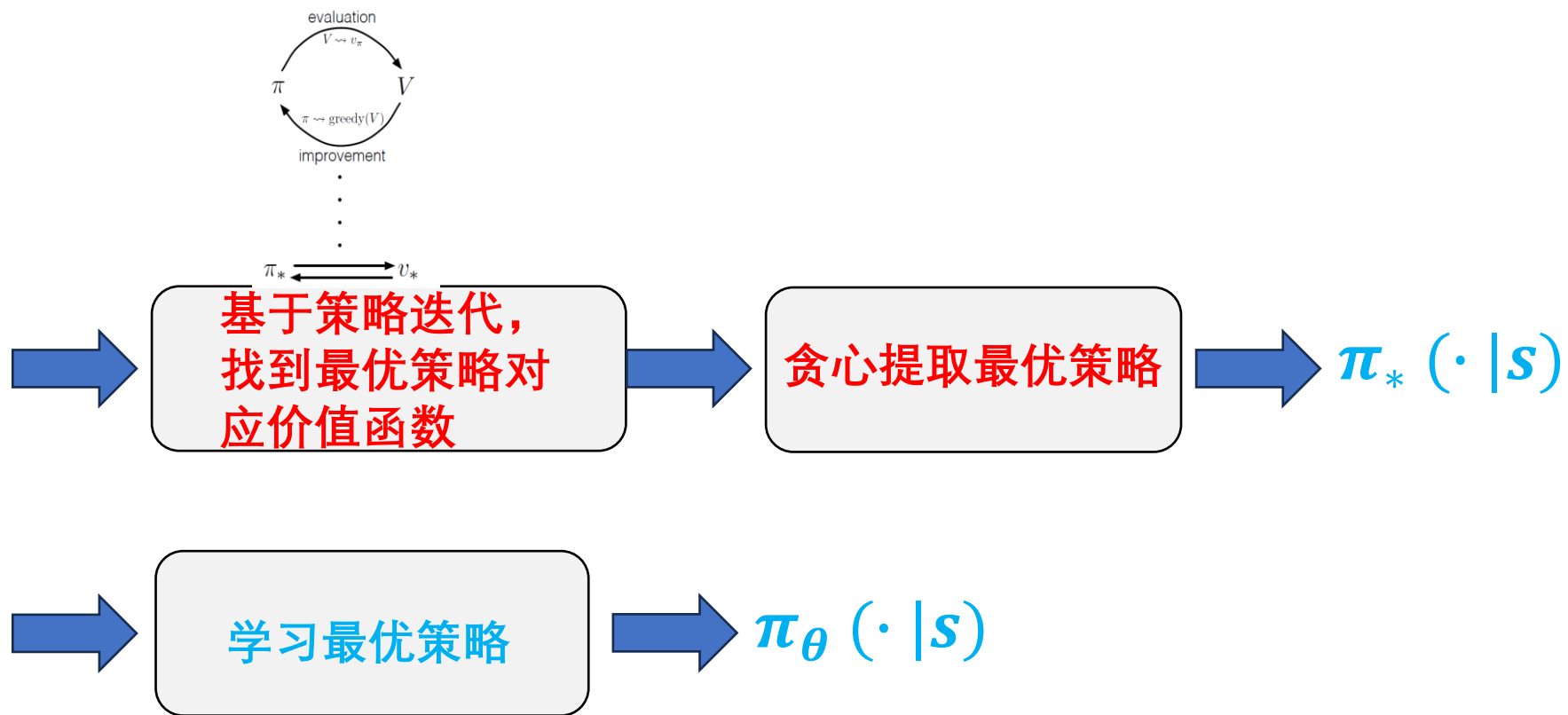
$$\pi_*(a|s) = \operatorname{argmax}_{a \in \mathcal{A}(s)} \sum_{s', r} p(s', r|s, a) \underline{[r + \gamma v_*(s')]}, \text{ for all } s \in \mathcal{S},$$

最优状态价值函数

动机



智能体



能否**避开**最优价值函数估计**这一中间步骤**，让智能体**直接学习**最优策略？

策略学习：策略梯度（Policy Gradient）方法

□策略学习：通过求解一个优化问题，学出最优策略函数或它的近似函数（比

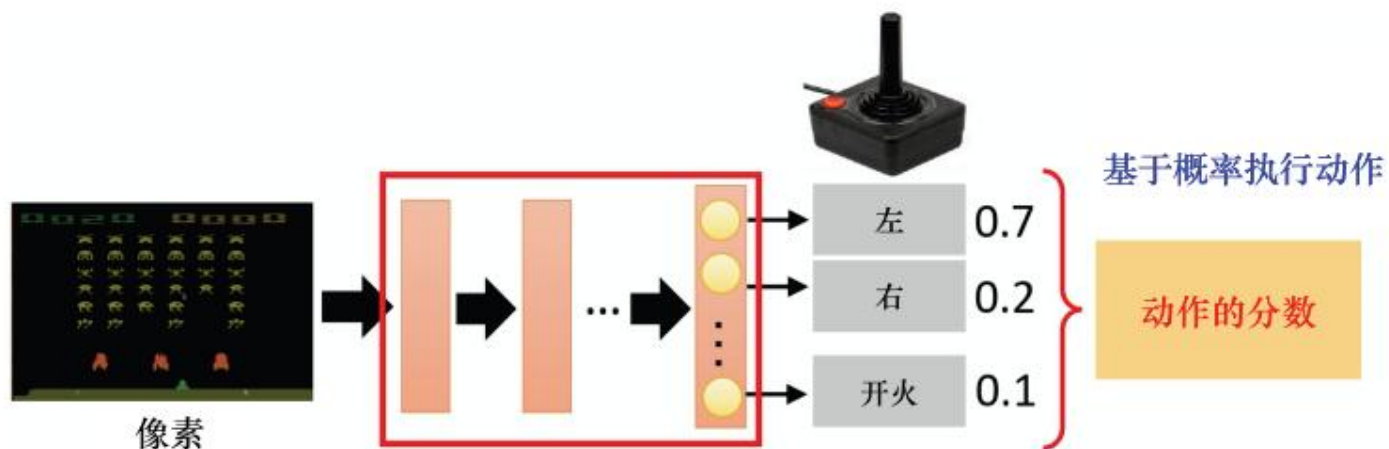
如策略网络 $\pi_{\theta}(\cdot)$) $\pi(a|s) \triangleq \mathbb{P}(A=a|S=s)$

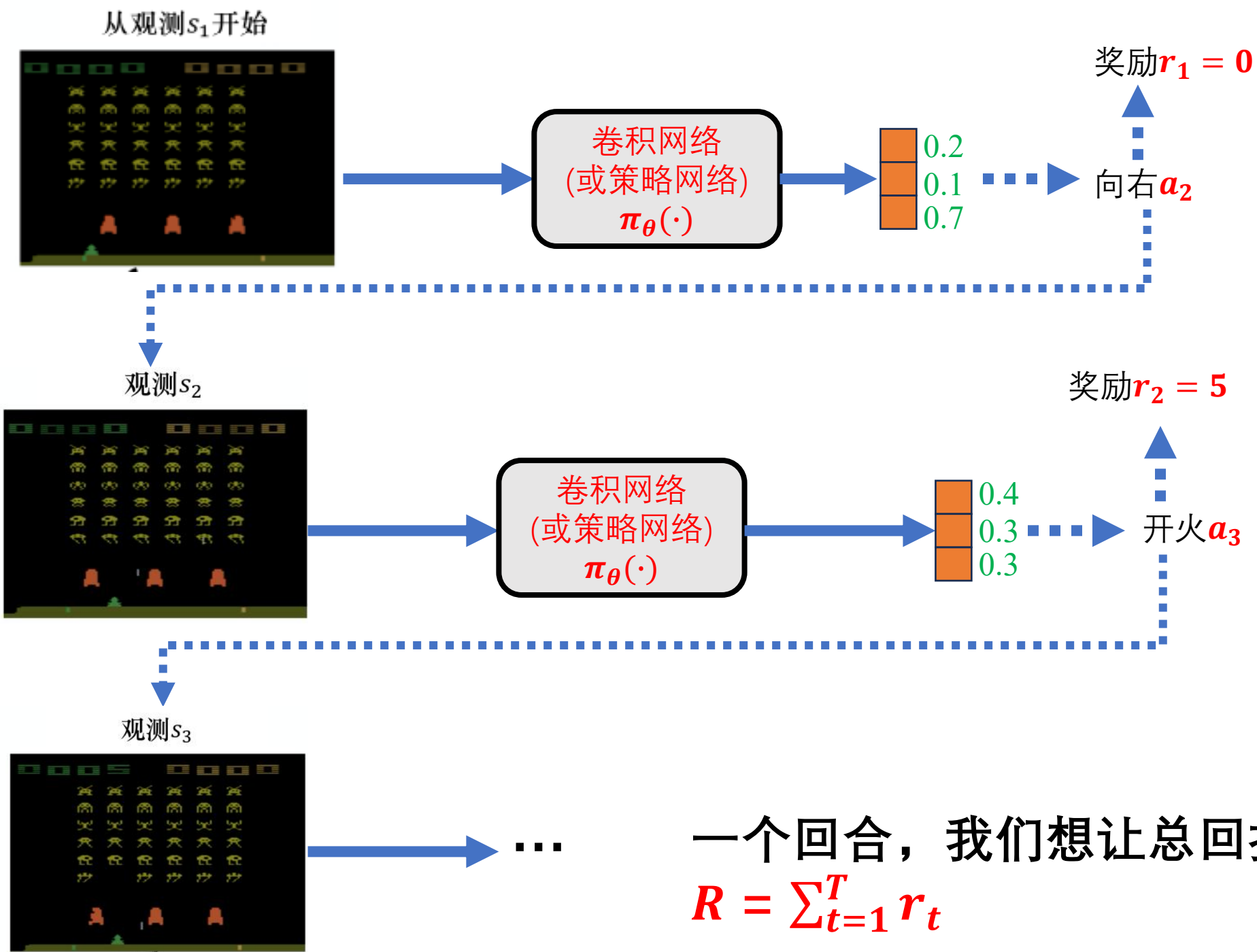
□策略梯度算法：避免了价值学习，直接学习状态到最优动作的映射关系

□本章假设：

➤ 动作空间是离散的，如 $\mathcal{A}=\{\text{上}, \text{下}, \text{右}\}$

➤ 策略函数 π 是个条件概率函数： $\pi_{\theta}(a|s) \triangleq \mathbb{P}(A=a|S=s)$





关于策略梯度，下面说法正确的是

- ☒ A 策略梯度方法学习的是最优策略函数
- ☐ B 策略梯度方法通过学习最优价值函数实现
- ☐ C 策略梯度学习的网络并不是最优策略函数

提交

强化学习的优化目标函数

$$\max_{\theta} J(\theta) = \mathbb{E}_{\tau \sim \pi_{\theta}} [\underline{R(\tau)}]$$

策略 轨迹回报

梯度? $\nabla_{\theta} J(\theta)$

交互多个回合，获取若干条轨迹，每条轨迹 τ 对应估算概率

智能体（ $\pi_{\theta}(\cdot)$ ）能获得的期望回报估计值

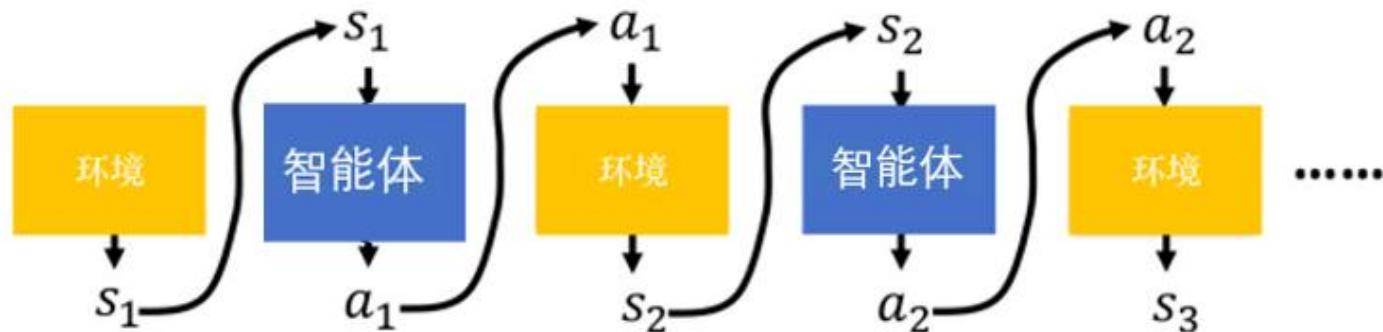
$$\mathbb{E}_{\tau \sim p_{\theta}(\tau)} [R(\tau)] = \sum_{\tau} R(\tau) p_{\theta}(\tau)$$

梯度上升: $\theta \leftarrow \theta + \alpha \nabla_{\theta} J(\theta)$ (最大化奖励 J)

梯度下降: $\theta \leftarrow \theta - \alpha \nabla_{\theta} L(\theta)$ (最小化损失 L)



策略梯度



智能体 $\pi_{\theta}(\cdot)$ 与环境交互生一个回合，产生一段轨迹

$$\tau = \{s_1, a_1, s_2, a_2, \dots, s_t, a_t\}$$

轨迹概率计算

$$p_{\theta}(\tau) = p(s_1) p_{\theta}(a_1|s_1) p(s_2|s_1, a_1) p_{\theta}(a_2|s_2) p(s_3|s_2, a_2) \dots$$

环境初始状态概率

$$= p(s_1) \prod_{t=1}^T p_{\theta}(a_t|s_t) p(s_{t+1}|s_t, a_t)$$

该轨迹对应总奖励（回报）为： $R(\tau)$

策略梯度推导

$$\bar{R}_\theta = \sum_{\tau} R(\tau) p_\theta(\tau)$$

优化 π_θ 最大化上述期望回报 (expected reward)

↓ 计算梯度

$$\nabla \bar{R}_\theta = \sum_{\tau} R(\tau) \nabla p_\theta(\tau)$$

$$= \sum_{\tau} R(\tau) p_\theta(\tau) \frac{\nabla p_\theta(\tau)}{p_\theta(\tau)}$$

$$= \sum_{\tau} R(\tau) p_\theta(\tau) \nabla \log p_\theta(\tau)$$

$$= \mathbb{E}_{\tau \sim p_\theta(\tau)} [R(\tau) \nabla \log p_\theta(\tau)]$$

关键一步：对数导数技巧

$$\frac{\nabla p_\theta(\tau)}{p_\theta(\tau)} = \nabla \log p_\theta(\tau)$$

采样 N 条 τ 并计算每条回报，计算每条轨迹对参数的梯度，累加求平均

$$\begin{aligned}\mathbb{E}_{\tau \sim p_{\theta}(\tau)} [R(\tau) \nabla \log p_{\theta}(\tau)] &\approx \frac{1}{N} \sum_{n=1}^N \underline{R(\tau^n) \nabla \log p_{\theta}(\tau^n)} \\ &= \frac{1}{N} \sum_{n=1}^N \sum_{t=1}^{T_n} \underline{R(\tau^n) \nabla \log p_{\theta}(a_t^n | s_t^n)}\end{aligned}$$

$$p_{\theta}(\tau) = p(s_1) \prod_{t=1}^T p_{\theta}(a_t | s_t) p(s_{t+1} | s_t, a_t)$$

$$\begin{aligned}\nabla \log p_{\theta}(\tau) &= \nabla \left(\log p(s_1) + \sum_{t=1}^T \log p_{\theta}(a_t | s_t) + \sum_{t=1}^T \log p(s_{t+1} | s_t, a_t) \right) \\ &= \nabla \log p(s_1) + \nabla \sum_{t=1}^T \log p_{\theta}(a_t | s_t) + \nabla \sum_{t=1}^T \log p(s_{t+1} | s_t, a_t) \\ &= \nabla \sum_{t=1}^T \log p_{\theta}(a_t | s_t) \\ &= \sum_{t=1}^T \nabla \log p_{\theta}(a_t | s_t)\end{aligned}$$

环境决定，与 θ 无关，梯度为0

梯度上升

$$\theta \leftarrow \theta + \eta \nabla \bar{R}_{\theta}$$

学习率

智能体与
状态-动作
对数据环
境交互，
收集

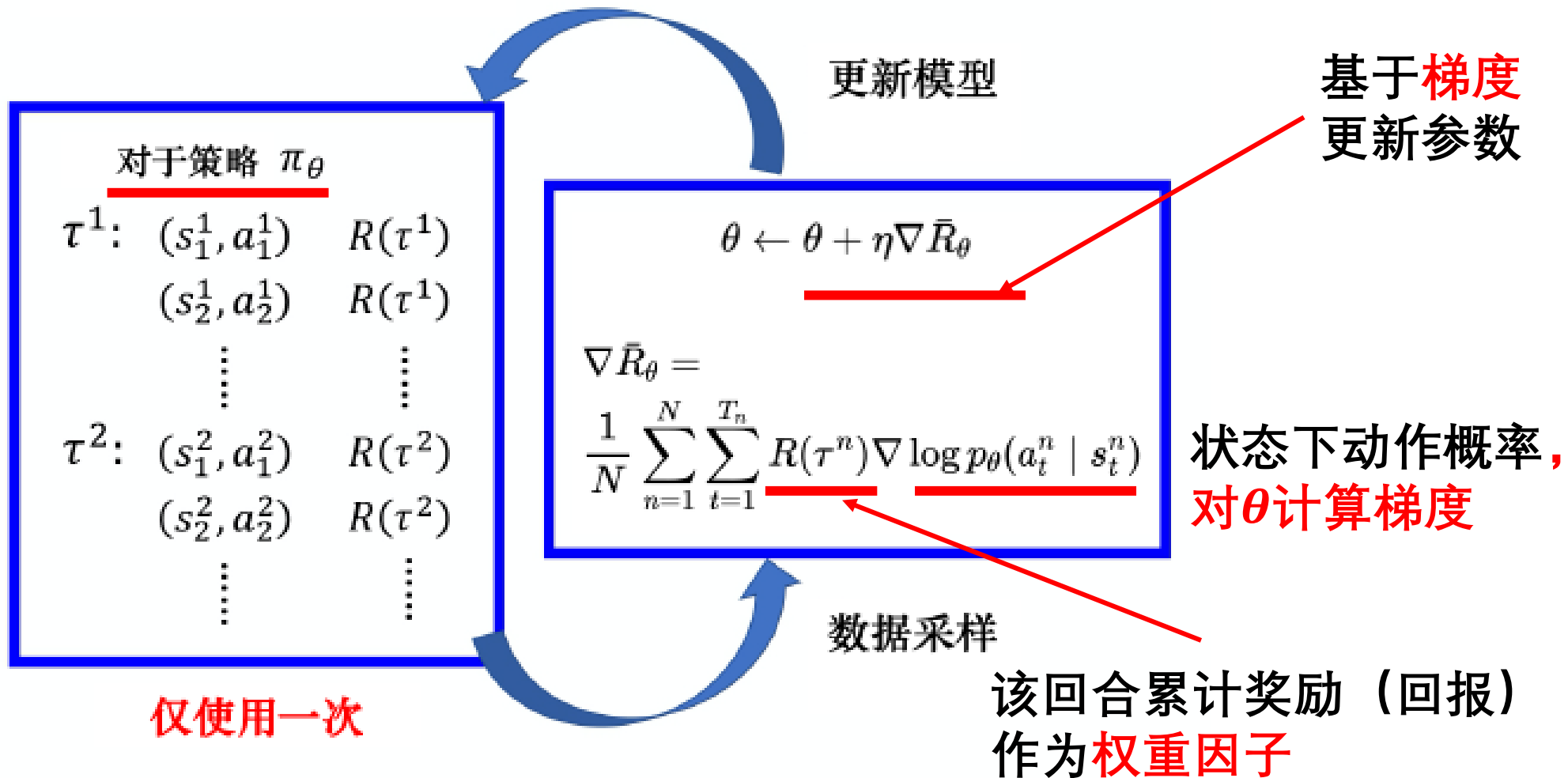


图 4.7 策略梯度

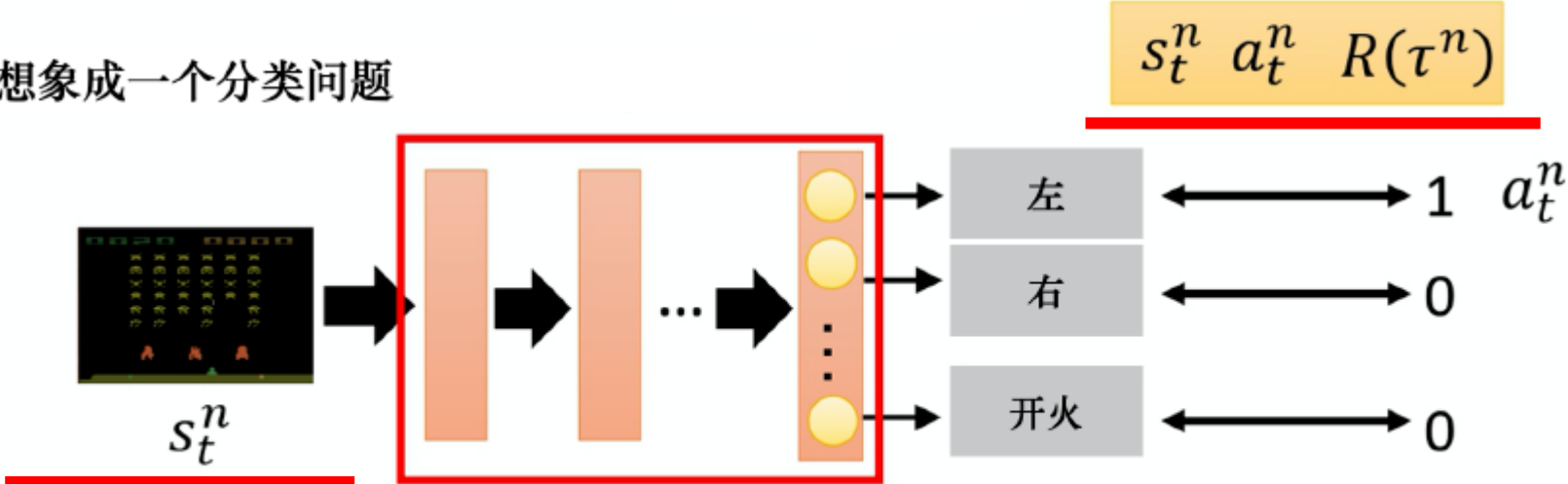
注意：采样的数据只会用一次，更新参数，丢掉数据，接着重新采样数据，再次更新参数...

策略梯度实现：分类介绍

执行

$$\theta \leftarrow \theta + \eta \nabla \bar{R}_\theta$$
$$\nabla \bar{R}_\theta = \frac{1}{N} \sum_{n=1}^N \sum_{t=1}^{T_n} R(\tau^n) \nabla \log p_\theta(a_t^n | s_t^n)$$

想象成一个分类问题



输入游戏一帧图像，利用神经网络（策略参数化）输出分类结果（执行哪一个动作）
目标函数：交叉熵损失

损失前面有整回合游戏的回报作为权重因子

$$\frac{1}{N} \sum_{n=1}^N \sum_{t=1}^{T_n} \log p_\theta(a_t^n | s_t^n) \longrightarrow \frac{1}{N} \sum_{n=1}^N \sum_{t=1}^{T_n} \underline{R(\tau^n)} \log p_\theta(a_t^n | s_t^n)$$

策略梯度和分类任务对比

$$\frac{1}{N} \sum_{n=1}^N \sum_{t=1}^{T_n} \log p_{\theta}(a_t^n | s_t^n) \xrightarrow{\text{PyTorch / TensorFlow.....}} \frac{1}{N} \sum_{n=1}^N \sum_{t=1}^{T_n} \nabla \log p_{\theta}(a_t^n | s_t^n)$$
$$\frac{1}{N} \sum_{n=1}^N \sum_{t=1}^{T_n} \underline{R(\tau^n)} \log p_{\theta}(a_t^n | s_t^n) \xrightarrow{\quad} \frac{1}{N} \sum_{n=1}^N \sum_{t=1}^{T_n} \underline{R(\tau^n)} \nabla \log p_{\theta}(a_t^n | s_t^n)$$

注意： PyTorch等框架支持自动求导梯度

策略梯度：梯度的一般表示形式

□ 在策略梯度中，可以把梯度写成下面这个更加一般的形式：

$$\nabla_{\theta} J(\theta) = \mathbb{E} \left[\sum_{t=0}^T \psi_t \nabla_{\theta} \log \pi_{\theta}(a_t | s_t) \right]$$

ψ_t 取如下形式：

1. $\sum_{t'=1}^T \gamma^{t'} r_{t'}$: 轨迹总回报

策略梯度： ψ_t 中添加基线

□ 给定状态 s 采取动作 a

- 若整场游戏奖励都是**正**，则要**增加** (s, a) 的概率
- 若整场游戏奖励是**负**，则要**减少** (s, a) 的概率
- 假设游戏的奖励都是正，最低是0，比如打乒乓球游戏，分数为0-21分，那么在训练时不管采取什么动作，则优化目标都是最大化动作概率

□ ψ_t 添加基线(b)

$$\nabla \bar{R}_\theta \approx \frac{1}{N} \sum_{n=1}^N \sum_{t=1}^{T_n} \underline{R(\tau^n) \nabla \log p_\theta(a_t^n | s_t^n)}$$

$$\nabla \bar{R}_\theta \approx \frac{1}{N} \sum_{n=1}^N \sum_{t=1}^{T_n} \underline{(R(\tau^n) - b) \nabla \log p_\theta(a_t^n | s_t^n)}$$

ψ_t 有正有负，正则增加
概率，负则减少概率

关于策略梯度中使用基线（baseline）优化，下面说法正确的是：

- ☒ A 增加基线，是为了防止所有奖励都为正
- ☒ B 增加基线，是为了防止所有奖励都为负
- ☒ C 回合累计奖励大于基线，则优化过程让动作概率提升
- ☐ D 回合累计奖励大于基线，则优化过程让动作概率降低

提交

策略梯度： ψ_t 考虑动作回报

$$\nabla \bar{R}_\theta \approx \frac{1}{N} \sum_{n=1}^N \sum_{t=1}^{T_n} \underbrace{(R(\tau^n) - b)}_{\text{动作 } a_t \text{ 之后的回报}} \nabla \log p_\theta(a_t^n | s_t^n)$$

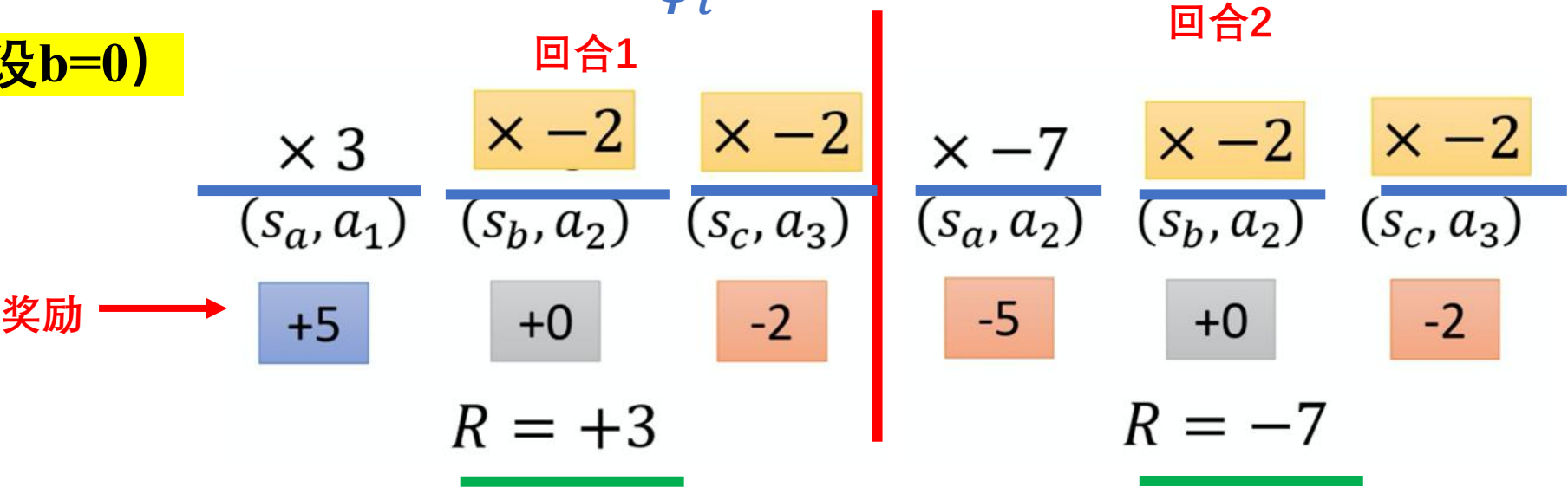
动作 a_t 之后的回报



一个回合里，所有动作-状态对的加权都是一样的，显然不太合理。游戏里面肯定有好动作，也有差动作

$$\nabla \bar{R}_\theta \approx \frac{1}{N} \sum_{n=1}^N \sum_{t=1}^{T_n} \underbrace{\left(\sum_{t'=t}^{T_n} r_{t'}^n - b \right)}_{\psi_t} \nabla \log p_\theta(a_t^n | s_t^n)$$

(假设 $b=0$)



策略梯度： ψ_t 考虑折扣回报

折扣回报

$$\nabla \bar{R}_\theta \approx \frac{1}{N} \sum_{n=1}^N \sum_{t=1}^{T_n} \underbrace{\left(\sum_{t'=t}^{T_n} \gamma^{t'-t} r_{t'}^n - b \right)}_{\psi_t} \nabla \log p_\theta (a_t^n \mid s_t^n)$$

策略梯度: REINFORCE (每一步动作计算对应的Q)

$$\nabla \bar{R}_\theta \approx \frac{1}{N} \sum_{n=1}^N \sum_{t=1}^{T_n} \underline{G_t^n} \underline{\nabla \log \pi_\theta(a_t^n | s_t^n)}$$

(假设b=0)

$$\begin{aligned} G_t &= \sum_{k=t+1}^T \gamma^{k-t-1} r_k \\ &= r_{t+1} + \gamma G_{t+1} \end{aligned}$$



$$(s_1, a_1, G_1), (s_2, a_2, G_2), \dots, (s_T, a_T, G_T)$$

策略梯度: REINFORCE

输入: 可微调的策略参数 $\pi(a|s, \theta)$ 。

算法参数: 步长大小 $\alpha > 0$ 。

初始化的策略参数 $\theta \in \mathbb{R}^{d'}$ (例如, 初始化为0)。

对每一个回合进行循环。

根据 $\pi(\cdot|\cdot, \theta)$, 生成 $s_0, a_0, r_0, \dots, s_{T-1}, r_T$ 。

对一个回合中的每一步进行循环, $t=0, 1, \dots, T-1$:

$$\begin{aligned} G &\leftarrow \sum_{k=t+1}^T \gamma^{k-t-1} r_k; & (G_t) \\ \theta &\leftarrow \theta + \alpha \gamma^t G \nabla \log \pi(a_t | s_t, \theta). \end{aligned}$$

(s_1, a_1, G_1)	对于每一个动作: 损失 = $-G_t \cdot \underbrace{[0, 1, 0]}_{\text{实际 } a_t} \cdot \log \underbrace{[0.2, 0.5, 0.3]}_{\pi(a s, \theta)}$
(s_2, a_2, G_2)	
$\dots$	
(s_T, a_T, G_T)	

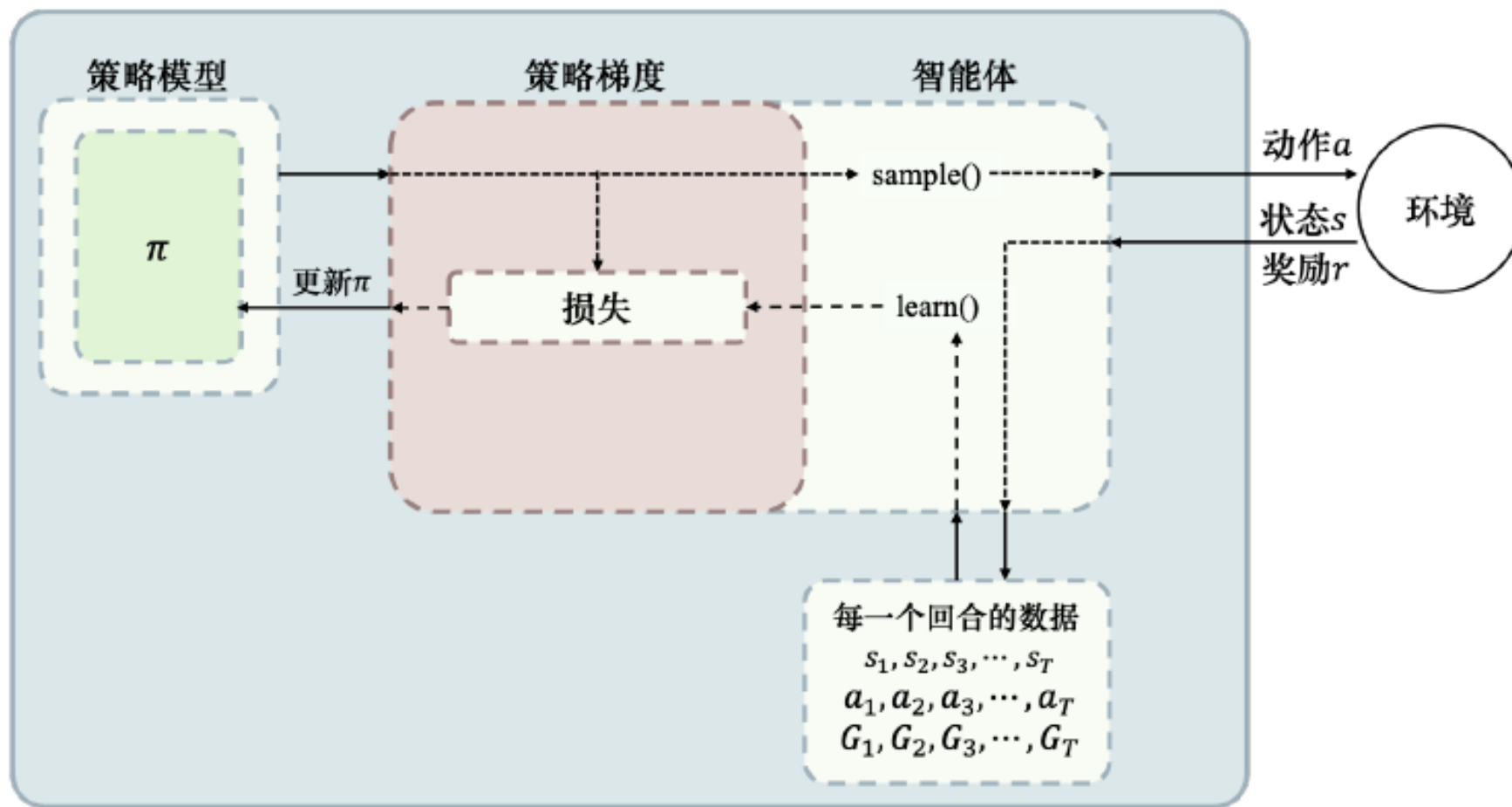


图 4.20 REINFORCE 算法示意

REINFORCE 算法基于蒙特卡洛采样，只能在轨迹序列结束后进行更新

案例：CartPole环境介绍

状态空间

维度	意义	最小值	最大值
0	车的位置	-2.4	2.4
1	车的速度	-Inf	Inf
2	杆的角度	$\sim -41.8^\circ$	$\sim 41.8^\circ$
3	杆尖端的速度	-Inf	Inf



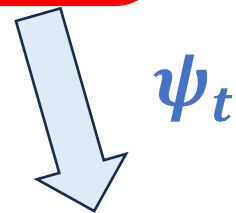
动作空间

标号	动作
0	向左移动小车
1	向右移动小车

策略梯度： ψ_t 用价值网络估计出来

$$\nabla \bar{R}_\theta \approx \frac{1}{N} \sum_{n=1}^N \sum_{t=1}^{T_n} \left(\sum_{t'=t}^{T_n} \gamma^{t'-t} r_{t'}^n - b \right) \nabla \log p_\theta (a_t^n | s_t^n)$$

b 通常可以由一个神经网络估计出来的



ψ_t

优势函数 (advantage function) $A_\theta(s_t, a_t)$

假设某状态 s_t 执行动作 a_t ，相较于其他可能的动作， a_t 相对来说多好，即**相对优势 (relative advantage)**。 $A_\theta(s_t, a_t)$ 可由一个网络（评论员 (critic) 网络）估计出来。

策略梯度: ψ_t 用价值网络估计出来

□ 优势函数: $A^{\pi_{\theta}}(s_t, a_t) = Q^{\pi_{\theta}}(s_t, a_t) - V(s_t)$

- 当前动作相对于平均水平的优势
- $Q^{\pi_{\theta}}(s_t, a_t)$: 动作价值函数

$$A^{\pi_{\theta}}(s_t, a_t) = Q^{\pi_{\theta}}(s_t, a_t) - V(s_t)$$

$$Q^{\pi_{\theta}}(s_t, a_t) = r_t + \gamma V^{\pi_{\theta}}(s_{t+1})$$

$$r_t + \gamma V^{\pi_{\theta}}(s_{t+1}) - V^{\pi_{\theta}}(s_t)$$

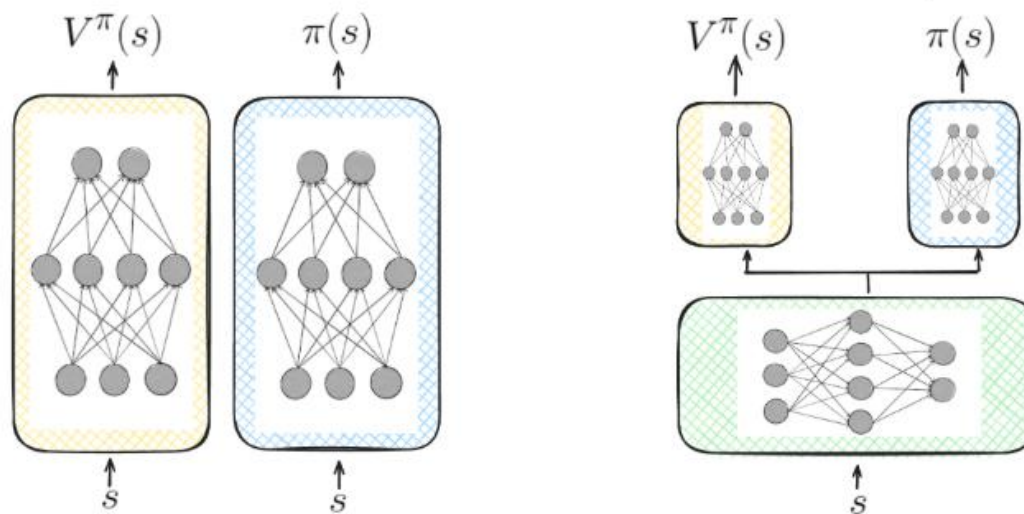
时序差分残差 (或(TD 误差))

Actor-Critic 算法借助价值网络估计动作价值，可以在每一步之后都进行更新

Actor-Critic 算法

□ Actor : 与环境交互, 产生交互回合数据

□ Critic: 基于Actor交互的数据学习一个价值函数。这个价值函数会用于判断在当前状态什么动作是好的, 什么动作不是好的, 进而帮助 Actor 进行策略更新。



策略网络 (Actor) : 输出动作的概率分布, 即“行为网络”。输出动作的概率分布, 即“行为网络”。
价值网络 (Critic) : 输出状态价值 $V(s)$, 用于评估当前状态的好坏。Critic 提供的 $V(s)$ 来计算 TD 残差。

Actor 更新

□ 策略网络 $\pi_{\theta}(s)$, 参数 θ

□ 策略梯度

$$\theta \leftarrow \theta + \alpha \cdot \frac{1}{N} \sum_{i=1}^N \sum_{t=0}^{T_i} \delta_t^{(i)} \cdot \nabla_{\theta} \log \pi_{\theta}(a_t^{(i)} | s_t^{(i)})$$

$$\psi_t = \delta_t = r_t + \gamma V(s_{t+1}) - V(s_t)$$

Critic 更新

□ Critic价值网络: $V_{\phi}(s)$, 参数 ϕ

- 目标网络: $V_{\phi^-}(s)$, 参数 ϕ^- (定期从 ϕ 复制, 不参与梯度计算)
- 从环境中采样得到转移数据 $(s_t, a_t, r_{t+1}, s_{t+1})$
- TD 目标 (不产生梯度): $y = r + \gamma V_{\phi^-}(s_{t+1})$, 其中 y 被视为固定目标, 不计算其对 ϕ 的梯度。

□ 价值网络的损失函数 (单个样本)

$$L(\phi) = \frac{1}{2}(y - V_{\phi}(s_t))^2 \text{ 或 } L(\phi) = \frac{1}{2}(r + \gamma V_{\phi^-}(s_{t+1}) - V_{\phi}(s_t))^2$$

□ 损失函数对 ϕ 的梯度为

$$\nabla_{\phi} L(\phi) = -(r + \gamma V_{\phi^-}(s_{t+1}) - V_{\phi}(s_t)) \nabla_{\phi} V_{\phi}(s_t)$$

Actor-Critic 伪代码

输入：折扣因子 γ ，策略学习率 α_θ ，价值网络学习率 α_ϕ ，目标网络更新周期 C

初始化：策略参数 θ （初始化策略 π_θ ） 价值网络参数 ϕ （初始化 V_ϕ ） 价值网络参数 ϕ^- （初始化 V_{ϕ^-} ）

目标价值网络参数 $\phi^- \leftarrow \phi$

重复（每个 episode）：

1. 重置环境，得到初始状态 s

2. 重复（直到 episode 终止）：

- 从当前策略采样动作： $a \sim \pi_\theta(\cdot|s)$
- 在环境中执行动作 a ，得到奖励 r 和下一状态 s'
- 将转移元组 (s, a, r, s') 存入临时缓冲区
- 如果 达到更新步数 或者 s' 是终止状态：
 - 对 缓冲区中每个转移 (s_t, a_t, r_t, s_{t+1}) 执行：

$$y_t = \begin{cases} r_t + \gamma \cdot V_{\phi^-}(s_{t+1}) & \text{若 } s_{t+1} \text{ 非终止} \\ r_t & \text{若 } s_{t+1} \text{ 终止} \end{cases}$$

$$A_t = y_t - V_\phi(s_t)$$

累加策略梯度: $\nabla_{\theta} \log \pi_{\theta}(a_t | s_t) \cdot A_t$

累加价值网络梯度: $\nabla_{\phi} (y_t - V_{\phi}(s_t))^2$

- 使用累加的梯度更新策略参数 θ (学习率 α_{θ})
- 使用累加的梯度更新价值网络参数 ϕ (学习率 α_{ϕ})
- 清空临时存储缓冲区
- 更新当前状态: $s \leftarrow s'$
- 每 C 步: 将目标价值网络参数软/硬更新 $\phi^{-} \leftarrow \phi$

输出: 优化后的策略 π_{θ}

策略梯度小结

□ 在策略梯度中，可以把梯度写成下面这个更加一般的形式：

$$\nabla_{\theta} J(\theta) = \mathbb{E} \left[\sum_{t=0}^T \psi_t \nabla_{\theta} \log \pi_{\theta}(a_t | s_t) \right]$$

ψ_t 有多种形式：

1. 轨迹总回报： $\sum_{t'=1}^T \gamma^{t'} r_{t'}$

2. 动作 a_t 之后的回报： $\sum_{t'=t}^T \gamma^{t'-t} r_{t'}$



REINFORCE 算法基于蒙特卡洛采样，只能在轨迹序列结束后进行更新

3. 基线版本的改进： $\sum_{t'=t}^T \gamma^{t'-t} r_{t'} - b(s_t)$

4. 动作价值函数： $Q^{\pi_{\theta}}(s_t, a_t)$

5. 优势函数： $A^{\pi_{\theta}}(s_t, a_t)$



优势函数 $Q^{\pi_{\theta}}(s_t, a_t) - V(s_t)$ (当前动作相对于平均水平的优势)



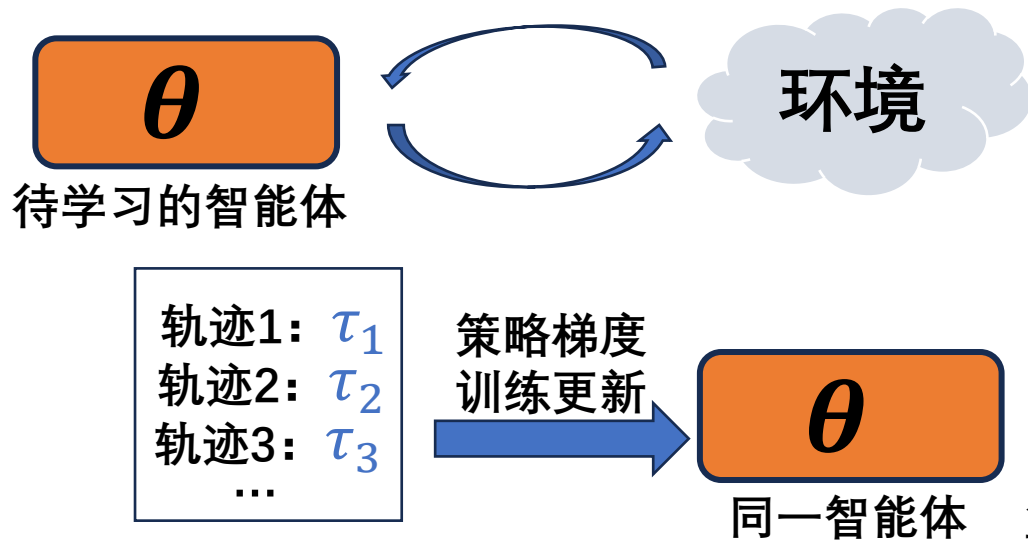
$$Q^{\pi_{\theta}}(s_t, a_t) = r_t + \gamma V^{\pi_{\theta}}(s_{t+1})$$

6. Actor-Critic: $r_t + \gamma V^{\pi_{\theta}}(s_{t+1}) - V^{\pi_{\theta}}(s_t)$ (时序差分残差 (或TD 误差))

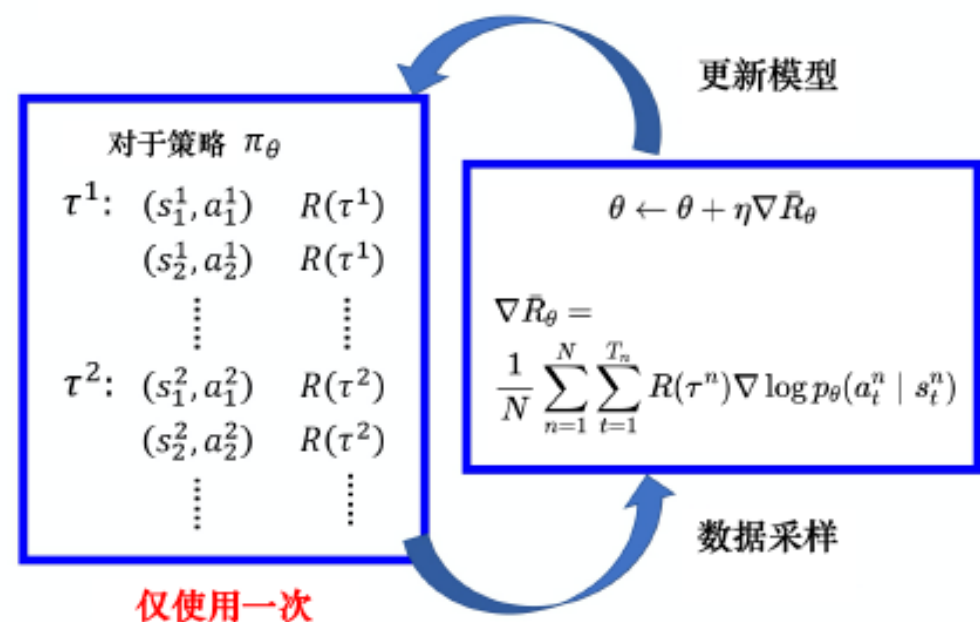
近端策略优化

(proximal policy optimization, PPO)

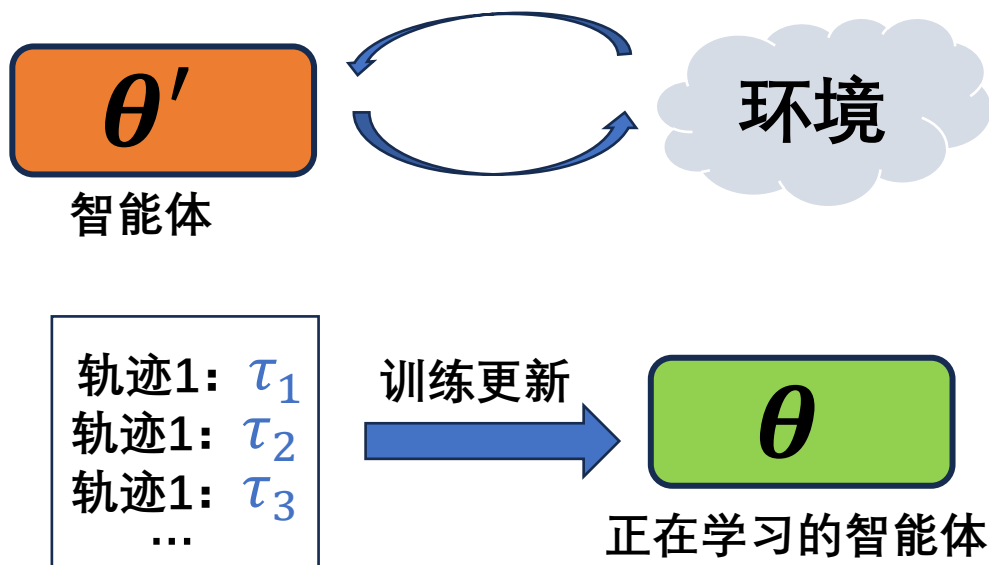
回顾：同策略和异策略



同策略



只能更新参数一次，一旦更新了参数，概率 $p_\theta(\tau)$ 就不对了。需重新采样数据，才能再次更新参数。这显然是非常花时间



异策略

用策略 $\pi_{\theta'}$ 与环境交互，用 θ' 采样到的数据去训练 θ 。可以多次使用 θ' 采样到的数据，多次执行梯度上升（gradient ascent）更新参数。

异策略理论基础：重要性采样 (importance sampling)

□ 随机变量 $X \sim P(X)$ ，则 $f(X)$ 的期望可通过从 $P(X)$ 中采样本点进行估算：

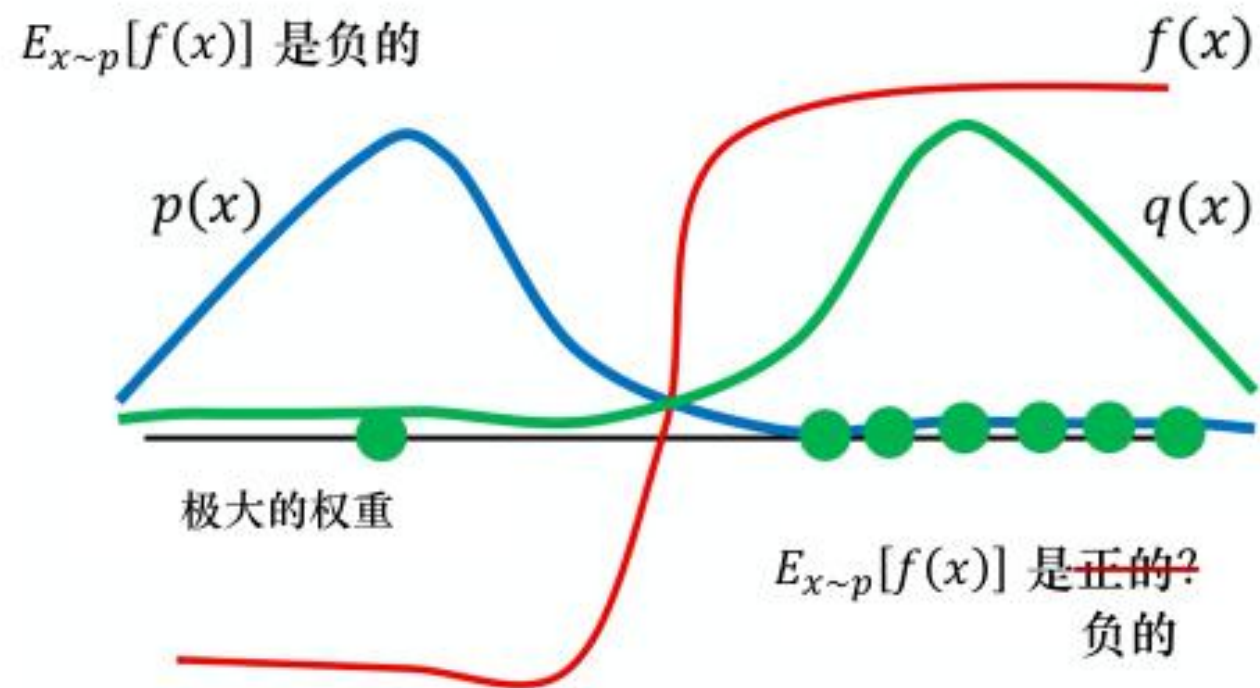
$$E[f(x)] = \int f(x)p(x)dx \approx \frac{1}{n} \sum_i f(x_i)$$

□ 若 $p(x)$ 较复杂，无法像均匀分布、正态分布一样直接采样。则借助重要性采样估计：

$$E[f(x)] = \int f(x)p(x)dx = \int f(x)\frac{p(x)}{q(x)}q(x)dx \approx \frac{1}{n} \sum_i f(x_i) \boxed{\frac{p(x_i)}{q(x_i)}} \quad \text{每个样本的重要度}$$

- $q(x)$ 是任意分布（且 $q(x)$ 不为 0），只需从分布 $q(x)$ 中采样 x ，估计其期望值。
- $p(x)/q(x)$ 被称为采样比或采样权重，用来抵消/校正从不同分布采样的概率差异。

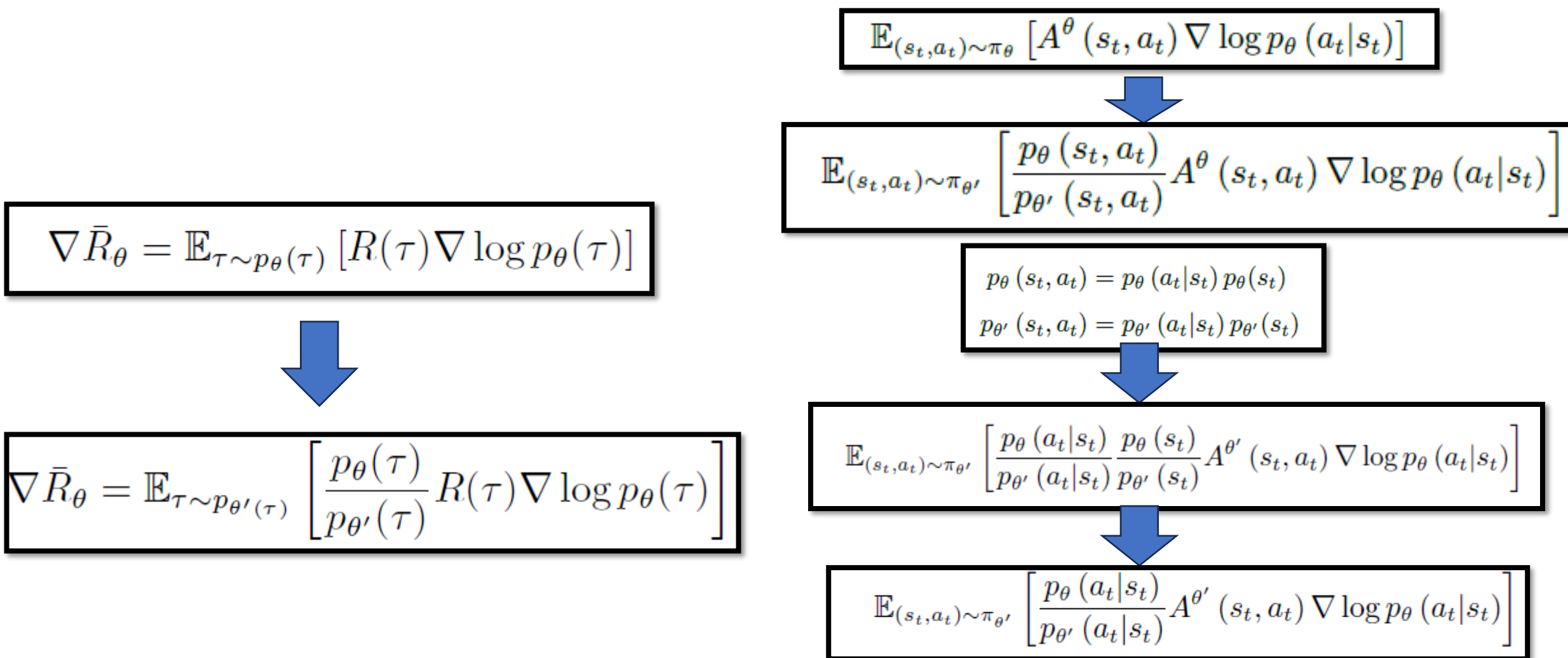
理论基础：重要性采样 (importance sampling)



如果采样次数不够多，当 $p(x)$ 和 $q(x)$ 差距很大时，则可能导致估计误差较大

异策略训练

- 基于重要性采样，可把同策略训练改成异策略训练。
- θ' 同环境交互获取数据，使用 θ' 而非 θ 进行交互有什么好处呢？
- θ 可多次优化，训练到一定程度， θ' 再重新做采样，这就是同策略换成异策略的妙处



为什么可以假设 $p_{\theta}(s_t)$ 和 $p_{\theta'}(s_t)$ 是一样的？

□一般来说，智能体看到状态往往与采取的动作是没有太大的关系的。

- 比如玩雅达利游戏，不同回合看到的游戏画面差不多，所以不同 θ 对 s_t 影响很小
- $p_{\theta}(s_t)$ 很难算， $p_{\theta}(s_t)$ 表示用 θ 去与环境交互， s_t 出现的概率，而这个概率很难算。
- $p_{\theta}(a_t|s_t)$ 很好计算， θ 是策略网络，输入状态 s_t ，该网络输出 a_t 概率，容易计算。

梯度

由梯度推导出优化的目标函数

$$\underbrace{\mathbb{E}_{(s_t, a_t) \sim \pi_{\theta'}} \left[\frac{p_{\theta}(a_t|s_t)}{p_{\theta'}(a_t|s_t)} A^{\theta'}(s_t, a_t) \right]}_{\nabla p_{\theta}(a_t|a_t)} \xrightarrow[\frac{p_{\theta}(a_t|s_t)}{p_{\theta'}(a_t|s_t)}]{\nabla f(x) = f(x) \nabla \log f(x)} J^{\theta'}(\theta) = \mathbb{E}_{(s_t, a_t) \sim \pi_{\theta'}} \left[\frac{p_{\theta}(a_t|s_t)}{p_{\theta'}(a_t|s_t)} A^{\theta'}(s_t, a_t) \right]$$

注意，对 θ 求梯度时， $p_{\theta'}(a_t|s_t)$ 和 $A^{\theta'}(s_t, a_t)$ 都是常数

近端策略优化 (PPO)

- 如果 $p_{\theta}(a_t|s_t)$ 与 $p_{\theta'}(a_t|s_t)$ 相差太多，即这两个分布相差太多，重要性采样的结果会较差。如何避免呢？
- PPO: 在训练时约束 θ 与 θ' 输出概率的一致性

约束本来要优化的项

约束 θ 与 θ' 输出的动作的KL 散度 (KL divergence)

$$J_{\text{PPO}}^{\theta'}(\theta) = J^{\theta'}(\theta) - \beta \text{KL}(\theta, \theta')$$
$$J^{\theta'}(\theta) = \mathbb{E}_{(s_t, a_t) \sim \pi_{\theta'}} \left[\frac{p_{\theta}(a_t | s_t)}{p_{\theta'}(a_t | s_t)} A^{\theta'}(s_t, a_t) \right]$$

PPO是同策略

- PPO (Proximal Policy Optimization) **本质上是同策略 (on-policy) 算法**，但它通过重要性采样和截断技巧，允许在一定程度上重复使用旧数据，也常被描述为“带有异策略 (off-policy) 行为的同策略方法”。
- **PPO 的目标函数**引入了重要性采样比率 $r_t(\theta)$ ，使得它可以利用旧策略生成的数据进行多次梯度更新。
- **但 PPO 会强制限制**新旧策略的差异（通过裁剪或 KL 惩罚），防止比率过大。这意味着它只能使用与**当前策略相近的旧数据**，不能像真正的异策略算法那样任意重用历史数据。
- **PPO 不是纯粹的异策略**。在实现上，它仍然需要定期用**当前策略**与环境交互采集新数据，然后在这批数据上多次更新。

PPO变种1：近端策略优化惩罚

□ 近端策略优化惩罚（PPO-penalty）

- 先初始化一个策略的参数 θ^0 。在每次迭代里面，我们用前一个训练迭代得到的演员参数 θ^k 与环境交互，采样到大量状态动作对。根据交互结果，估算 $A^{\theta^k}(s_t, a_t)$ 。
- 和原来的策略梯度不同，原来的策略梯度只能更新一次参数，更新完需要重新采样。现在我们用 θ^k 交互，采样这组数据之后，可以让 θ 更新很多次最大化目标函数。
- 由于重要性采样的理论基础，即使这些状态-动作对是从 θ^k 采样得到也没有关系。

$$J_{\text{PPO}}^{\theta^k}(\theta) = J^{\theta^k}(\theta) - \beta \text{KL}(\theta, \theta^k)$$

$$J^{\theta^k}(\theta) \approx \sum_{(s_t, a_t)} \frac{p_{\theta}(a_t | s_t)}{p_{\theta^k}(a_t | s_t)} A^{\theta^k}(s_t, a_t)$$

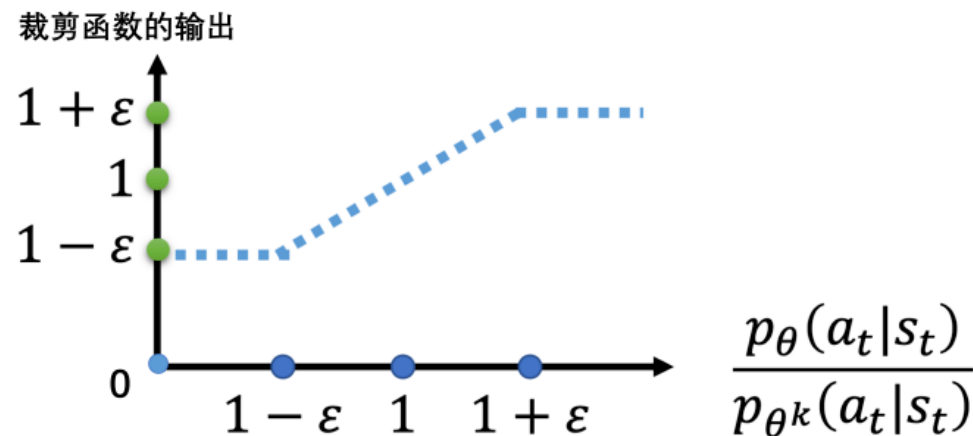
PPO变种2：近端策略优化裁剪

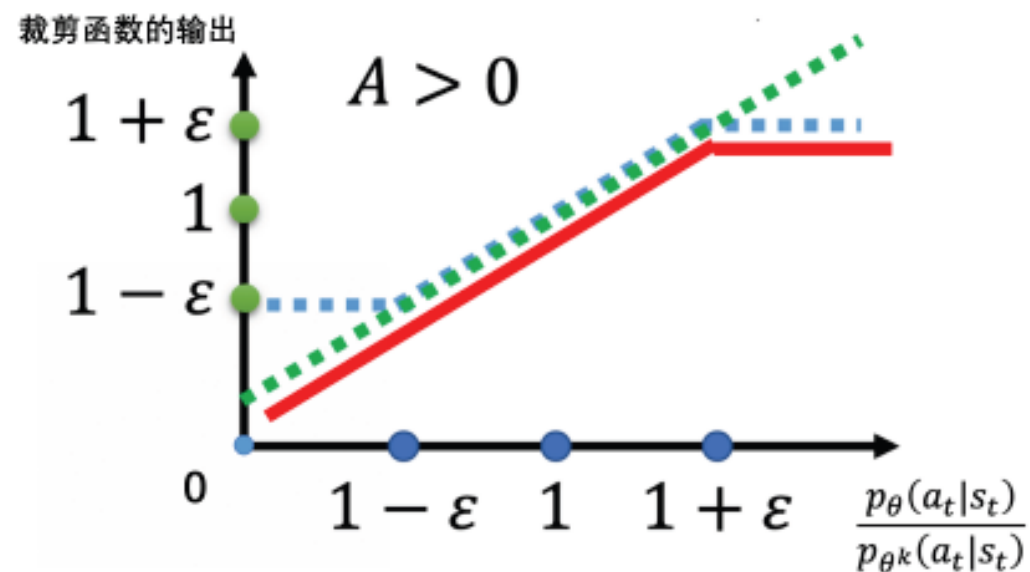
□ 近端策略优化裁剪的目标函数里没有KL 散度，其要最大化的目标函数为

$$J_{\text{PPO2}}^{\theta^k}(\theta) \approx \sum_{(s_t, a_t)} \min \left(\frac{p_{\theta}(a_t|s_t)}{p_{\theta^k}(a_t|s_t)} A^{\theta^k}(s_t, a_t), \right. \\ \left. \text{clip} \left(\frac{p_{\theta}(a_t|s_t)}{p_{\theta^k}(a_t|s_t)}, 1 - \varepsilon, 1 + \varepsilon \right) A^{\theta^k}(s_t, a_t) \right)$$

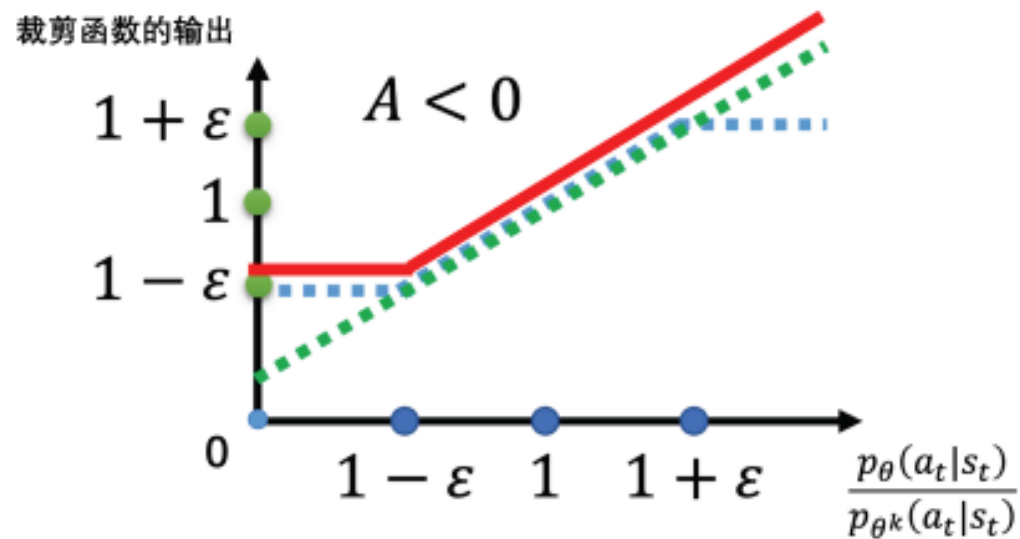
超参

$$\varepsilon = 0.2 \quad \text{clip} \left(\frac{p_{\theta}(a_t|s_t)}{p_{\theta^k}(a_t|s_t)}, 0.8, 1.2 \right)$$





(a) $A > 0$



(b) $A < 0$

图 5.3 A 对裁剪函数输出的影响

总结

- 策略梯度
- PPO

课程回顾

第一章：RL背景与重要术语

第二章：多臂老虎机：探索与利用

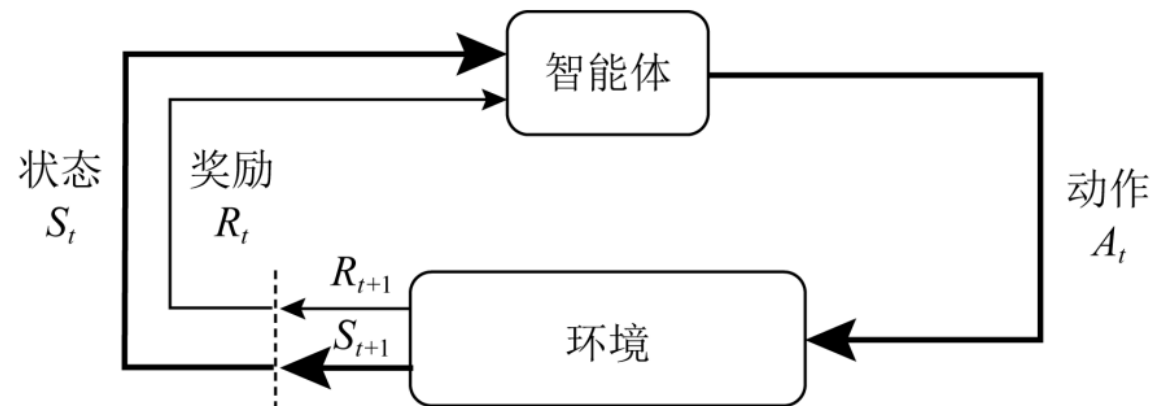
第三章：MDP：RL分析问题的一种工具

第四章：动态规划：价值/策略迭代

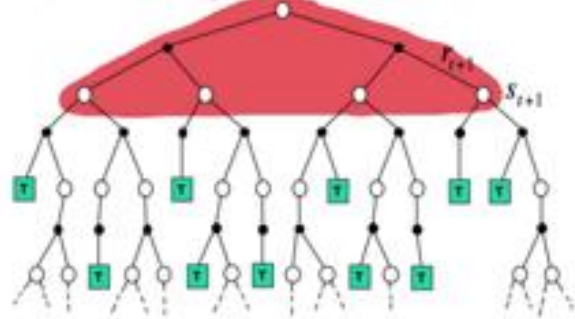
第五章：蒙特卡洛方法：估计状态/动作价值

第六章：时序差分学习：估计状态/动作价值

第七章：【高阶】策略梯度：直接学习策略

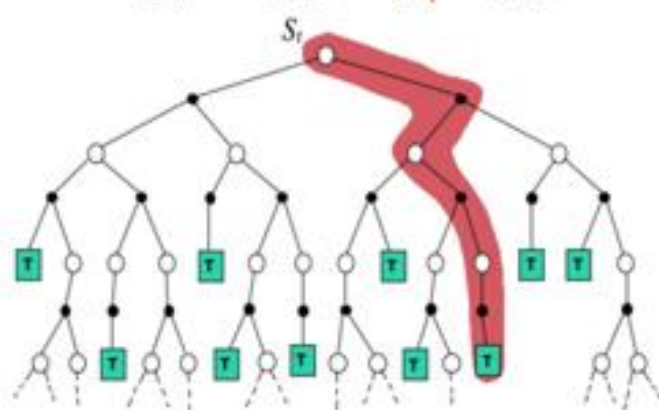


$$V(s) \leftarrow \sum_a \pi(a|s) \left(r(s, a) + \gamma \sum_{s'} p(s'|s, a) V(s') \right)$$



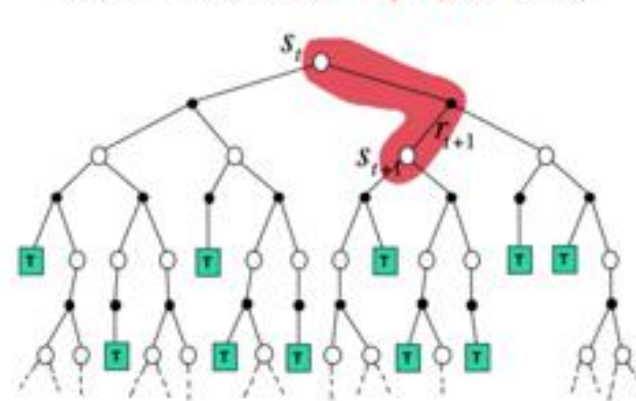
DP

$$V(s) \leftarrow V(s) + \alpha (G_t - V(s))$$



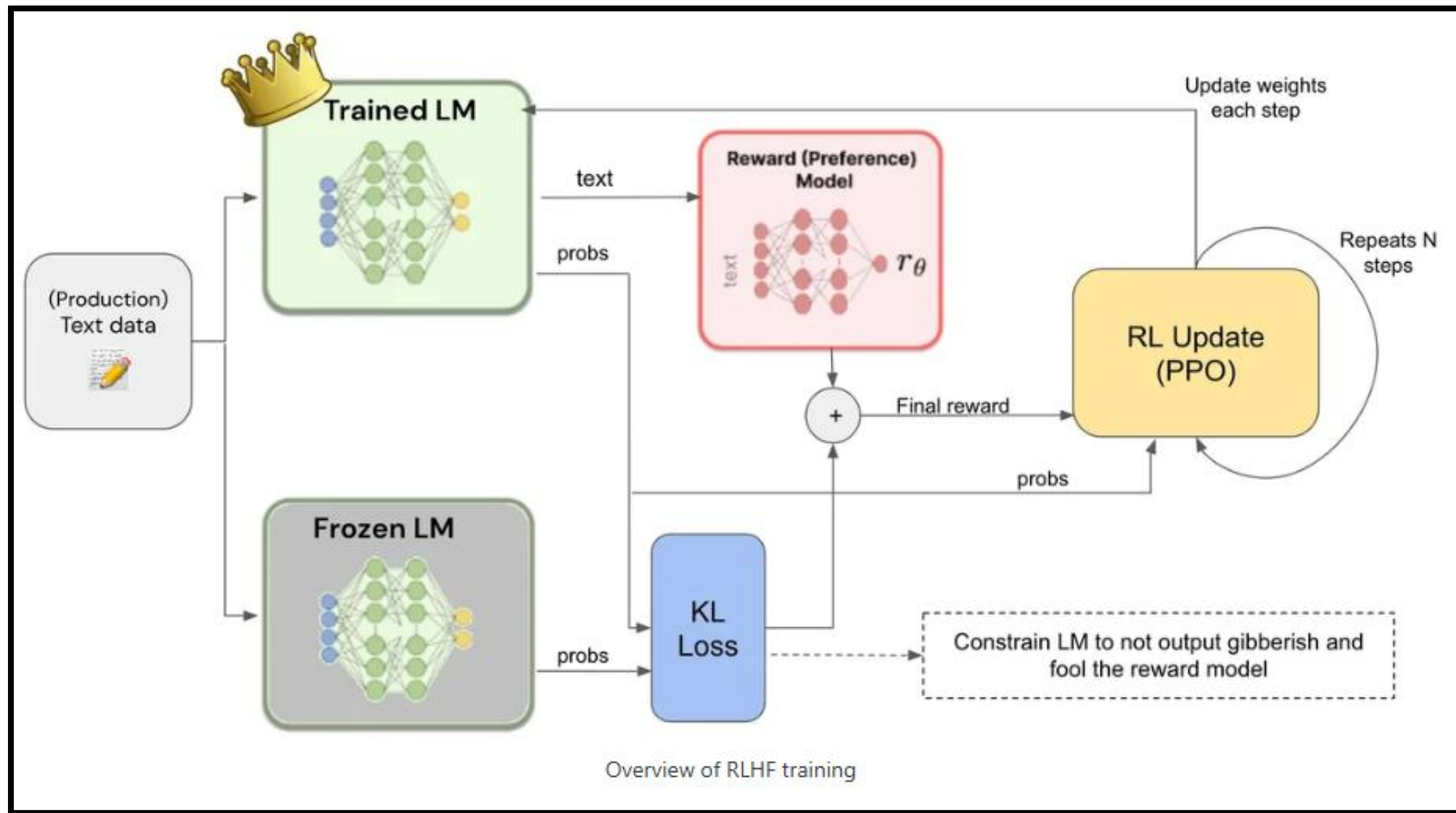
MC

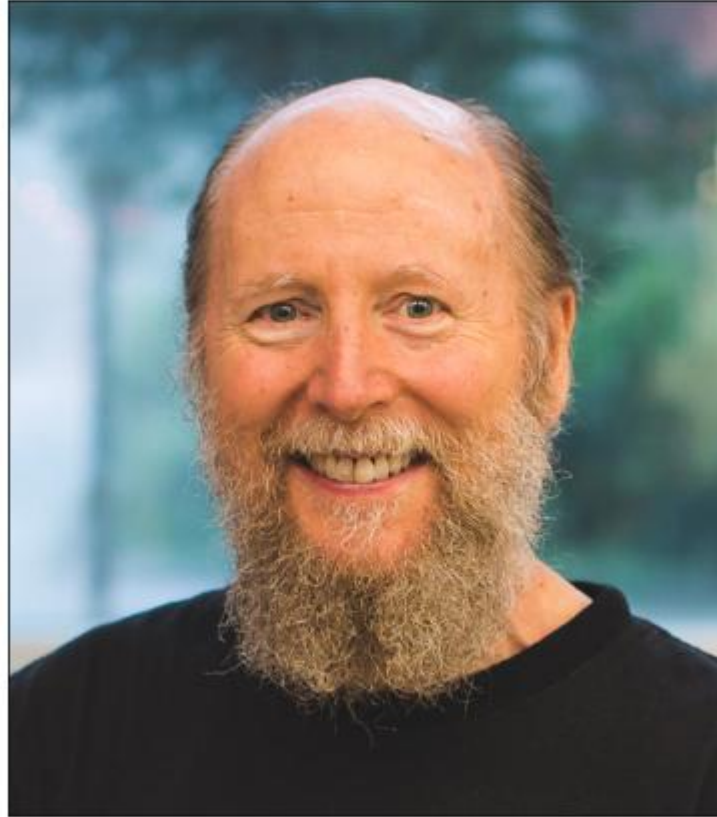
$$V(s) \leftarrow V(s) + \alpha (r' + \gamma V(s') - V(s))$$



TD(0)

更多内容【由于课时限制未涉及】...





2024年ACM A.M. 图灵奖授予了**Andrew G. Barto**和**Richard S. Sutton**，以表彰他们在强化学习（Reinforcement Learning, RL）领域的开创性贡献。他们从20世纪80年代开始，通过一系列论文奠定了强化学习的理论基础，并开发了关键算法。强化学习是构建智能系统的重要方法之一。

强化学习是

一种重要的分析解决问题的工具

一种思维方式

一种哲学

强化学习结课报告

□ 结课报告（有latex模版，下午两点前会发到QQ群里）

➤ 课程内容总结（用自己的语言，不要简单复制ppt内容，最好图文并茂）

➤ 个人展望思考

□ 截止日期：5月3号晚上11:59前。

感谢大家